

Dynamic Multi-Head CNC Execution Middleware (DMHC-EM)

Software Design Specification (SDS)

Volume 1 – Project Vision, Problem Definition and System Specification

Version: 1.0

Document Type: Software Design Specification

Project Status: Research & Development

Target Language: Python

Development Environment: Visual Studio Code

Target Machine: Multi-Head CNC Machine

Chapter 1 — Executive Summary

Introduction 1.1

Computer Numerical Control (CNC) machines are among the most important manufacturing technologies in modern industry. They are capable of transforming digital designs into precise .physical products through automated tool movement controlled by G-code instructions

Despite the significant advances in CNC hardware, the majority of commercially available CNC systems still rely on a single cutting head executing one machining path at a time. This sequential execution model limits productivity because only one region of the workpiece is .processed at any given moment

As production demands increase, manufacturers seek methods to reduce machining time .without sacrificing dimensional accuracy or requiring entirely new manufacturing processes

.This project proposes a software-based solution rather than a hardware redesign

Instead of modifying the CAM software or rewriting CNC firmware, this project introduces an .intelligent middleware positioned between the CAM-generated G-code and the CNC controller

The middleware analyzes the original machining program, reconstructs its geometry, partitions the workload dynamically across multiple CNC heads, generates multiple synchronized G-code programs, and guarantees that the final machined part is geometrically identical to the original .single-head execution

The proposed system aims to improve productivity through safe parallel machining while
.preserving complete compatibility with existing CAD/CAM workflows

Vision 1.2

The long-term vision of this project is to create a universal software layer capable of transforming conventional single-head CNC programs into optimized multi-head machining .programs without requiring modifications to CAD software, CAM software, or CNC controllers

The system should become machine-independent, controller-independent, and CAM-independent

.Any standard G-code generated by existing CAM software should be accepted as input

Mission 1.3

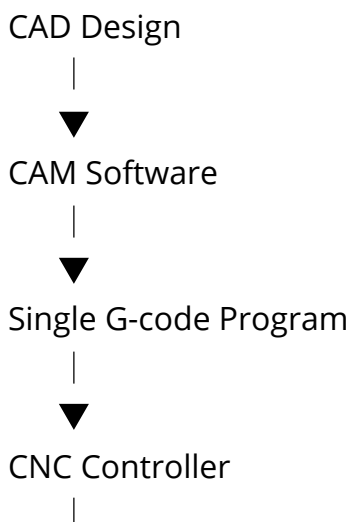
:Design and implement an intelligent software middleware that

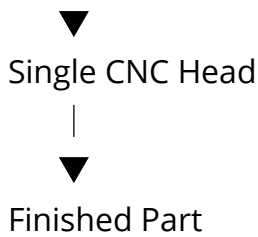
- .Receives one standard G-code file
- .Understands the machining process
- .Reconstructs toolpaths
- .Partitions machining intelligently
- .Prevents collisions
- .Generates multiple executable G-code files
- .Preserves machining accuracy
- .Reduces total machining time

Chapter 2 — Background

Conventional CNC Workflow 2.1

:The current manufacturing workflow is typically





.The CAM software computes a single continuous machining strategy

.The CNC controller executes every command sequentially

.Only one cutting head performs machining

.No parallelism exists

Limitation of Current Workflow 2.2

Although modern CNC machines are extremely precise, their machining strategy is
.fundamentally sequential

Even when multiple regions of a workpiece are completely independent, the machine still
.processes them one after another

This creates idle opportunities where multiple cutting heads could theoretically operate
.simultaneously

The bottleneck is therefore not always the machine mechanics, but rather the software
.workflow

Chapter 3 — Problem Statement

:The central engineering problem addressed by this project is

How can a standard single-head G-code program be transformed into multiple synchronized G-code programs that execute in parallel without changing the final machined geometry

.The solution must satisfy several strict constraints

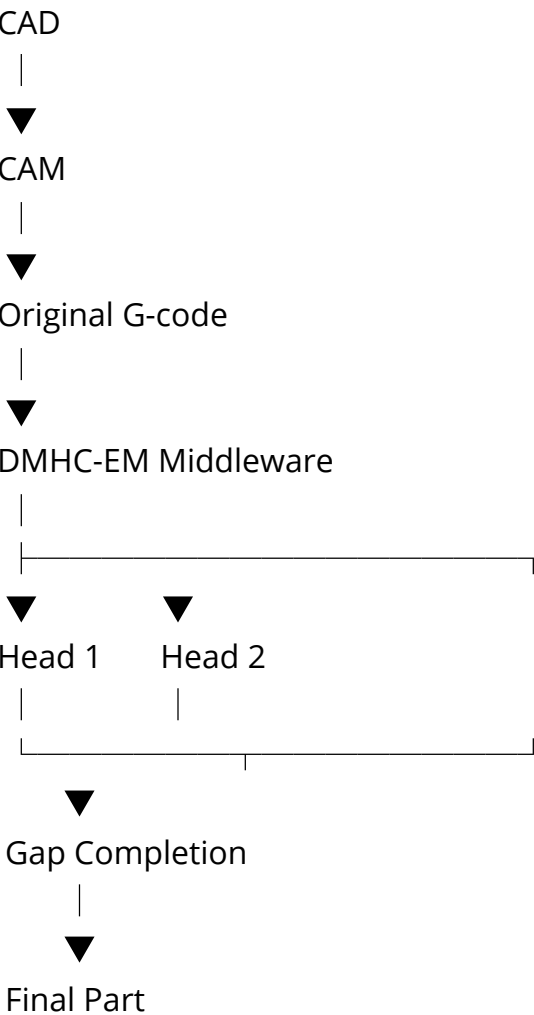
:The generated programs must

- .Never collide •
- .Never duplicate machining •
- .Never omit machining •
- .Never alter the final geometry •
- .Never violate machine safety •

The only acceptable difference between the original execution and the new execution is reduced
.machining time

Chapter 4 — Proposed Solution

.Instead of modifying CAM software, a new software layer will be introduced



.This middleware becomes responsible for every optimization step

Chapter 5 — Research Objectives

Primary Objective

Develop an intelligent middleware capable of converting one G-code file into three coordinated
.G-code programs while preserving identical machining results

Secondary Objectives

:The software shall

- .Parse standard G-code •
- .Reconstruct machining toolpaths •

- .Analyze machining density
- .Compute dynamic X-axis partitions
- .Create a configurable mechanical safety gap
- .Prevent physical collisions
- .Generate independent executable G-code programs
- .Validate geometric equivalence
- .Reduce total machining time

Chapter 6 — Design Philosophy

.Several important design decisions define this project

Deterministic System 6.1

.The software must always generate identical output for identical input

.No randomness is allowed

Controller Independence 6.2

:The middleware should remain independent from

- GRBL
- LinuxCNC
- Mach3
- Mach4

.Only standard G-code should be assumed whenever possible

CAM Independence 6.3

:The software must not depend on

- Fusion 360
- SolidWorks CAM
- Mastercam
- Aspire
- VCarve

.Any compliant G-code should be accepted

Geometry Preservation 6.4

:The geometry produced after executing

+

Head 2

+

Gap Completion

.must be mathematically identical to executing the original G-code

.This is the single most important design requirement

Chapter 7 — Functional Requirements

:The system shall

FR-1

.Load one G-code file

FR-2

.Parse every supported command

FR-3

.Track complete machine state

FR-4

.Reconstruct continuous toolpaths

FR-5

.Identify machining entities

FR-6

.Estimate machining workload

FR-7

.Perform dynamic workload partitioning

FR-8

.Insert configurable mechanical gap

FR-9

.Generate three independent G-code files

FR-10

.Guarantee safe parallel execution

FR-11

.Generate gap completion program

FR-12

.Validate machining equivalence

FR-13

.Generate execution report

Chapter 8 — Non-Functional Requirements

:The software shall provide

- High reliability •
- Deterministic execution •
- Extensible architecture •
- Modular implementation •
- Clear documentation •
- Unit testing •
- Maintainability •
- Performance suitable for large G-code files •

Chapter 9 — Project Scope

Included

- 2D G-code •
- G0 •
- G1 •
- Dynamic X partitioning •
- Mechanical safety gap •
- Parallel scheduling •
- Independent program generation •
- Validation engine •

Excluded (Version 1)

- G2/G3 arcs
- axis machining-5
- Tool changes
- Multiple tools
- Automatic feed optimization
- Real-time controller integration

Chapter 10 — Success Criteria

:The project will be considered successful if

- .1 .A valid G-code file is accepted as input
- .2 .Three executable G-code files are generated
- .3 .Files 1 and 2 can execute in parallel without entering the mechanical gap
- .4 .File 3 completes only the excluded region after parallel execution finishes
- .5 .No collisions occur
- .6 .No duplicated cuts occur
- .7 .No missing cuts occur
- .8 .The final machined geometry is identical to the original
- .9 .The total machining time is reduced compared with the original single-head execution
- .10 .The software architecture allows future extension to more than two parallel heads

Volume 2 – Mechanical Model, Coordinate System & System Constraints

Dynamic Multi-Head CNC Execution Middleware (DMHC-EM)
Software Design Specification (SDS)
Version 1.0

Chapter 1 — Mechanical System Overview

Physical Interpretation of the System 1.1

The proposed system assumes a **multi-head CNC machine** where multiple independent cutting heads operate on the same workpiece simultaneously

.Each head is mechanically constrained to move within a defined region of the workspace

The system does not modify hardware behavior; instead, it imposes a **software-defined spatial partitioning layer** on top of the existing machine workspace

Multi-Head CNC Concept 1.2

:A standard CNC machine has

- One spindle
- One tool center point (TCP)
- One active machining region

:The proposed system assumes

- Multiple independent spindles (Head 1, Head 2, ... Head N)
- Each head has its own logical workspace region
- All heads share the same physical machine frame

Chapter 2 — Coordinate System Definition

Global Coordinate System 2.1

:The system uses a standard Cartesian coordinate system

- X-axis → primary partitioning axis
- Y-axis → machining depth direction (unchanged by partitioning)
- Z-axis → tool engagement axis (up/down motion)

Partitioning Axis Selection 2.2

.The system **only performs workload partitioning on the X-axis**

:Reason

- Most CAM toolpaths have strong spatial locality on X
- Simplifies collision avoidance
- Enables deterministic partitioning
- Reduces geometric complexity

Coordinate Space Assumption 2.3

:All toolpaths are assumed to exist in a normalized 2D plane

- Z is constant during cutting (2.5D machining assumption)
- No simultaneous multi-axis interpolation (for v1)

Chapter 3 — Mechanical Workspace Model

Workspace Definition 3.1

:The CNC workspace is defined as

$$X \in [0, W]$$

$$Y \in [0, H]$$

:Where

- W = total machine width •
- H = total machine height •

Multi-Head Partition Model 3.2

:For a 2-head system

$$|----- \text{Zone A} -----| --- \text{GAP} --- |----- \text{Zone B} -----|$$

:Each zone is defined as

- Zone A → Head 1 active region •
- Zone B → Head 2 active region •
- GAP → Mechanical safety buffer (inactive region) •

Chapter 4 — Mechanical GAP Definition

Definition 4.1

.The **Mechanical GAP** is a software-defined exclusion zone along the X-axis

:It is defined as

- A non-machining region •
- A collision prevention boundary •
- A physically enforced separation between heads •

Mathematical Definition 4.2

:Let

- WWW = machine width •
- ggg = gap width •
- xcx_cxc = center of partition •

:Then

$$\text{Gap} = [x_c - g/2 , x_c + g/2]$$

Functional Role of GAP 4.3

:The GAP ensures

- No simultaneous tool presence in the same physical region •
- No overlap between Head 1 and Head 2 toolpaths •
- :Safe mechanical clearance for •
 - tool radius ○
 - backlash ○
 - vibration ○
 - positioning error ○

Critical Engineering Requirement 4.4

.The GAP is NOT optional

.It is a **hard constraint** in scheduling

:Any toolpath intersecting the GAP must be

- either removed from parallel execution •
- or moved to the GAP completion phase (Head 3 / File 3) •

Chapter 5 — Machine Configuration Model

Machine Parameters 5.1

:The system defines a configuration object

MachineConfig:

width (W)
 height (H)
 number_of_heads (N)
 gap_width (g)
 tool_diameter
 safety_margin

Default Assumptions (Initial Prototype) 5.2

:The system assumes

- Machine width = 1.0 m •
- Machine height = 1.0 m •
- Gap width = configurable (default 0.1 m) •
- Number of heads = 2 (extensible to N) •
- Tool diameter = unknown (user-defined later) •

Important Design Note (Critical) 5.3

.The system MUST NOT hardcode machine dimensions

:Instead

- All physical parameters are configurable
- Default values are only placeholders
- Final system behavior depends on runtime configuration

Chapter 6 — Mechanical Safety Constraints

Collision Model 6.1

:A collision occurs when

- Two tool centers occupy overlapping X regions
- Or tool radius intersects forbidden region (GAP)

Safety Rules 6.2

Rule 1 — Zone Isolation

:Head 1

$X \in \text{Zone A only}$

:Head 2

$X \in \text{Zone B only}$

Rule 2 — GAP Exclusion

:No head is allowed to generate motion inside

Gap region

Rule 3 — Temporal Separation

- Head 1 and Head 2 may run in parallel
- Head 3 (Gap Fill) must run ONLY after both finish

Chapter 7 — Workload Representation Model

Toolpath Abstraction 7.1

.Raw G-code is NOT used directly

:It is converted into

Toolpath → Segment Graph → Contours → Regions

Segment Definition 7.2

:A segment is defined as

- Start point (X1, Y1)
- End point (X2, Y2)
- Feed rate
- Motion type (G0 / G1)

Geometry Projection 7.3

:All segments are projected onto X-axis

$\text{Projection}(\text{segment}) = [\min(X1, X2), \max(X1, X2)]$

.This projection is used for partitioning only

Chapter 8 — Partitioning Constraints

Partition Principle 8.1

.Partitioning is NOT geometric splitting

.It is a **workload distribution problem**

Valid Partition Conditions 8.2

:A valid partition must ensure

- Every segment belongs to exactly one zone
- No duplication of machining
- No missing toolpaths
- No intersection with GAP
- No violation of contour integrity (where applicable)

Invalid Partition Conditions 8.3

:A partition is invalid if

- It splits a closed contour incorrectly
- It causes discontinuous machining without re-entry logic
- It creates unreachable tool states
- It modifies geometry

Chapter 9 — System Behavior Model

Execution Model 9.1

:The system produces

File 1 → Head 1 (Zone A)

File 2 → Head 2 (Zone B)

File 3 → Gap Completion (Zone C)

Temporal Execution Logic 9.2

Head 1 and Head 2 execute in parallel .1

System waits for both completion .2

Head 3 executes remaining geometry .3

Final part is considered complete .4

Chapter 10 — Design Principle Summary

:The entire mechanical model is based on four principles

Spatial Separation 10.1

.Each head has a dedicated X-range

Temporal Synchronization 10.2

.Gap fill occurs after parallel execution

Deterministic Partitioning 10.3

.Same input → same partition output

Geometry Preservation 10.4

.Final output must match original G-code exactly

Volume 3 — Software Architecture

Dynamic Multi-Head CNC Execution Middleware (DMHC-EM)

Software Design Specification (SDS)

Version 1.0

Chapter 1 — Architectural Overview

System Philosophy 1.1

:The system is designed as a **compiler-like middleware** that transforms

G-code (low-level machine instructions)



Intermediate Representation (IR)



Optimized Multi-Head Execution Plan

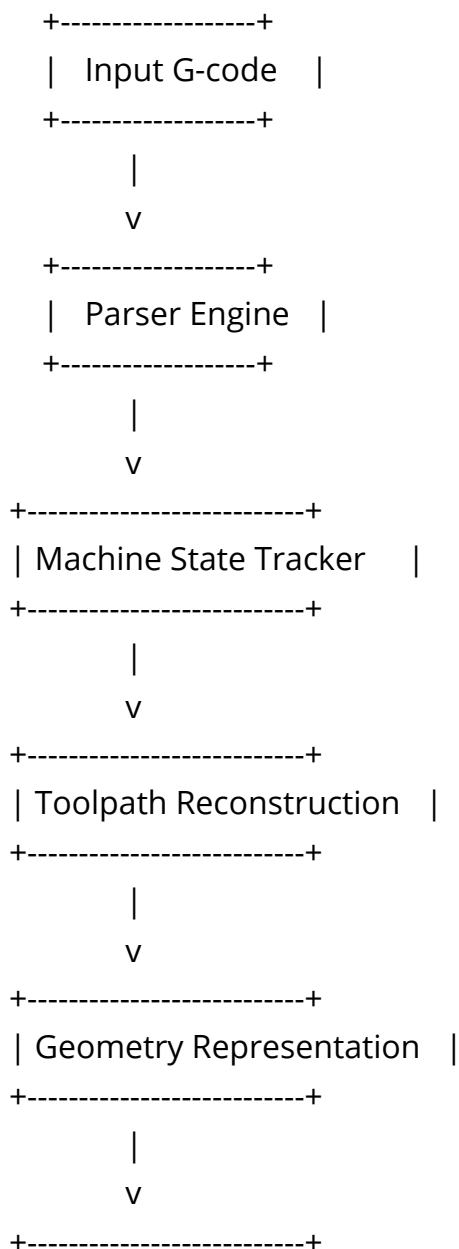


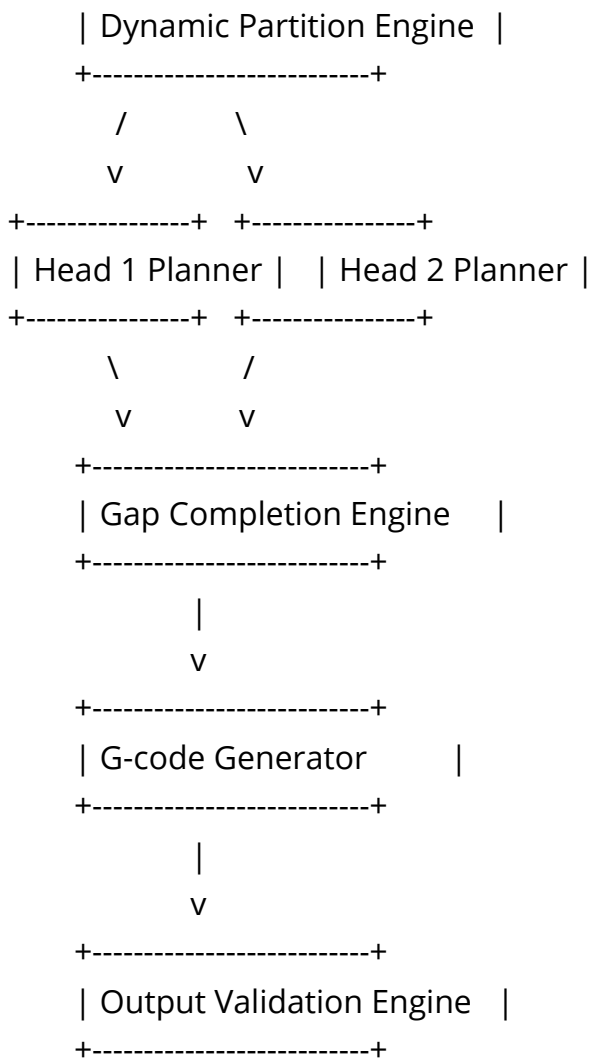
3 Independent G-code Programs

.This means the system is NOT a simple splitter

.It is a **motion-planning compiler for CNC systems**

High-Level Architecture 1.2





Chapter 2 — Core Design Principle

Compiler Analogy 2.1

:The system behaves like a compiler

Compiler Stage	CNC System Equivalent
Lexer	G-code Parser
AST	Toolpath Graph
Optimization	Partition Engine
Codegen	G-code Generator

Key Insight 2.2

.The system does NOT manipulate G-code directly

.It manipulates a **structured geometric representation**

Chapter 3 — Internal Data Model (IR)

Intermediate Representation (Critical) 3.1

:Every G-code file is converted into

IR = Toolpath Graph

IR Structure 3.2

:Core Entities

Point .1

(x, y, z)

Segment .2

Segment:

start_point

end_point

motion_type (G0 / G1)

feed_rate

Toolpath .3

Toolpath:

ordered list of segments

continuity constraints

contour type (open / closed)

Region .4

Region:

bounding_box

density_score

assigned_head

MachineState .5

:Tracks modal commands

G90 / G91

G21

current_position

spindle_state

feed_rate

Chapter 4 — Module Architecture

Package Structure (Python) 4.1

DMHC_EM/

|

|—— core/

|

|—— parser.py

```
| |—— machine_state.py
| |—— ir_builder.py
|
|—— geometry/
| |—— segment.py
| |—— toolpath.py
| |—— contour.py
| |—— region.py
|
|—— analysis/
| |—— density_analyzer.py
| |—— projection_engine.py
| |—— workload_estimator.py
|
|—— partition/
| |—— dynamic_splitter.py
| |—— gap_manager.py
| |—— scheduler.py
|
|—— generation/
| |—— gcode_generator.py
| |—— head_builder.py
| |—— gap_filler.py
|
|—— validation/
| |—— equivalence_checker.py
| |—— collision_detector.py
| |—— execution_validator.py
|
|—— simulation/
| |—— cnc_simulator.py
|
|—— main.py
```

Chapter 5 — Module Responsibilities

Parser Module 5.1

:Responsibilities

- Read raw G-code
- Tokenize instructions
- Detect motion commands

- Build MachineState updates •
- Forward commands to IR builder •

IR Builder 5.2

:Converts flat instructions into

- segments •
- toolpaths •
- contours •

Geometry Engine 5.3

:Responsibilities

- Convert segments into spatial objects •
- Compute bounding boxes •
- Detect closed loops •
- Identify machining regions •

Density Analyzer 5.4

:Computes

Workload(X) = amount of cutting per X interval

:Outputs

- heatmap along X-axis •
- density clusters •
- sparse vs dense regions •

Dynamic Partition Engine 5.5

.Core module

:Responsibilities

- Decide optimal X split •
- Balance workload between heads •
- Respect GAP constraint •
- Prevent contour breaking when unsafe •

Gap Manager 5.6

:Responsibilities

- Define mechanical exclusion zone •

- Ensure no head enters GAP
- Collect leftover geometry for Head 3

Scheduler 5.7

:Responsibilities

- Assign toolpaths to heads
- Ensure parallel safety
- Ensure temporal ordering

G-code Generator 5.8

:Responsibilities

- Convert IR back to G-code
- Ensure machine state correctness
- Add safe moves (G0, Z lifts)
- Ensure standalone execution

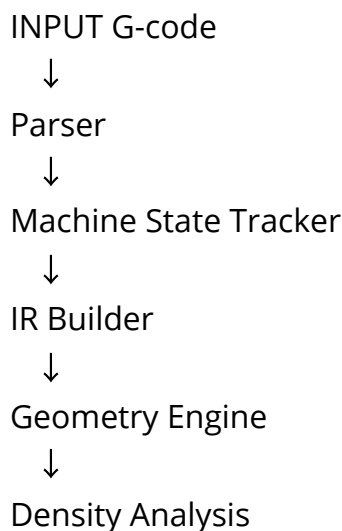
Validation Engine 5.9

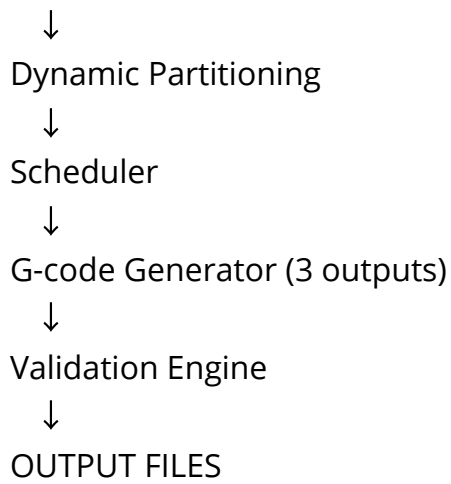
:Responsibilities

- Check geometry equivalence
- Detect missing segments
- Detect duplication
- Simulate execution
- Validate safety constraints

Chapter 6 — Execution Pipeline

Full Data Flow 6.1





Output Files 6.2

:The system always produces

head1.gcode → Zone A
head2.gcode → Zone B
gapfill.gcode → Zone C (after completion)

Chapter 7 — Critical Design Constraints

Determinism 7.1

.Same input → same output always

.No randomness allowed

Geometry Preservation 7.2

:IR must guarantee

Head1 + Head2 + GapFill = Original Toolpath

No Direct G-code Splitting 7.3

:The system MUST NOT

- split text lines •
- copy-paste G-code •

.It must regenerate everything from IR

Safety First 7.4

:No output is valid if

- collision risk exists
- GAP is violated
- toolpath continuity is broken

Chapter 8 — Extensibility Model

:The architecture is designed to support

- Multi-head > 2
- 3D machining (future)
- Arc support (G2/G3)
- Tool changes
- Adaptive feed optimization
- Real-time streaming

Chapter 9 — Design Summary

:The system is fundamentally

A geometry-aware CNC compiler with parallel execution optimization

.not a splitter

Volume 4 — G-code Parsing & Toolpath Reconstruction Engine

Dynamic Multi-Head CNC Execution Middleware (DMHC-EM)
Software Design Specification (SDS)
Version 1.0

Chapter 1 — Core Idea of This Volume

Why this module exists 1.1

.Raw G-code is NOT a geometric representation

:It is only

- movement commands
- modal states
- machine instructions

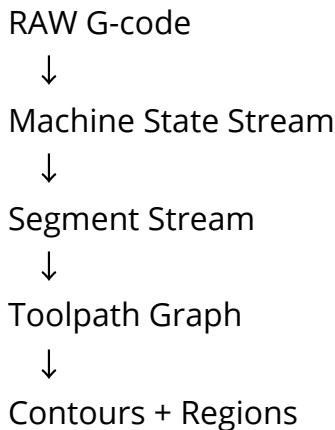
:Example

G1 X10 Y10
G1 X20 Y10
G1 X20 Y20

:This looks simple, but internally it represents
a continuous geometric path, not independent lines

Key Transformation 1.2

:This module converts



Chapter 2 — Parsing Philosophy

Incorrect Approach (What NOT to do) 2.1

- Splitting by lines ✗
- Splitting by X value ✗
- Copy-pasting commands ✗
- Treating each G1 as independent ✗

.These lead to broken geometry (as we already saw)

Correct Approach 2.2

- Interpret machine state ✓
- Track continuous motion ✓
- Build geometric segments ✓
- Reconstruct paths ✓

Chapter 3 — Machine State Tracking

Machine State Model 3.1

:Every G-code instruction updates a state machine

MachineState:

position (x, y, z)
feed_rate
spindle_state
distance_mode (G90/G91)
units (G21/G20)
motion_mode (G0/G1)

State Evolution Example 3.2

G90
G0 X0 Y0
G1 X10 Y0

:State transitions

- G90 → absolute mode •
- G0 → rapid move (non-cutting) •
- G1 → cutting move •

Important Insight 3.3

A segment is NOT a line in the file
A segment is a transition between machine states

Chapter 4 — Segment Construction

Segment Definition 4.1

:A segment is defined as

Segment:

start_point
end_point
type (cut / rapid)
feed_rate
tool_state

Segment Generation Rule 4.2

:Every motion command generates

- geometric segment 1 •

:BUT ONLY after resolving

- absolute vs relative mode •
- modal state •
- previous position •

Example Conversion 4.3

:Input

```
G0 X0 Y0  
G1 X10 Y0  
G1 X10 Y10
```

:Output

```
S1: (0,0) → (10,0) CUT  
S2: (10,0) → (10,10) CUT
```

Chapter 5 — Toolpath Construction

Definition 5.1

:A toolpath is

A continuous ordered list of segments

Toolpath Rules 5.2

:A toolpath is split when

- tool lifts (Z move) •
- spindle stops •
- rapid move occurs (G0) •
- discontinuity exists •

Toolpath Example 5.3

Toolpath 1:

S1 → S2 → S3 → S4 (continuous contour)

Toolpath 2:

S5 → S6 → S7 (new contour)

Chapter 6 — Contour Detection

Closed Contours 6.1

:A contour is closed if

`start_point == end_point`

:Example

:Square

- (0,0) •
- (10,0) •
- (10,10) •
- (0,10) •
- (0,0) •

Open Contours 6.2

:Not closed loops

- engraving paths •
- tool sweeps •
- partial cuts •

Why Contours Matter 6.3

.This is CRITICAL for your project

:Because

You cannot split a closed contour randomly without breaking geometry

Chapter 7 — Toolpath Graph Representation

Graph Model 7.1

:We convert toolpaths into a graph

Nodes = points

Edges = segments

Paths = toolpaths

?Why Graph 7.2

:Because partitioning is

a graph partitioning problem, not a text problem

$G = (V, E)$

V = set of coordinates

E = motion segments

Chapter 8 — Spatial Projection (X-axis mapping)

Projection Function 8.1

:Each segment is projected onto X-axis

$\text{Projection}(\text{Segment}) = [\min(x_1, x_2), \max(x_1, x_2)]$

Purpose 8.2

:Used for

- workload estimation
- partition decision
- density map generation

Important Warning 8.3

.Projection is ONLY for analysis

It must NOT modify geometry ❌

It is used only for decision-making ✔

Chapter 9 — Density Model Construction

Work Density Function 9.1

:We define

$D(x)$ = machining effort per X interval

Interpretation 9.2

- high $D(x)$ → dense machining
- low $D(x)$ → sparse machining

Why this matters 9.3

:This directly determines

- where Head 1 works
- where Head 2 works
- where GAP is placed

Chapter 10 — Reconstruction Principle

Critical Rule 10.1

:After reconstruction

.The system must be able to regenerate the EXACT original G-code geometry from IR

Lossless Requirement 10.2

:The transformation

G-code → IR → Toolpaths → G-code

:must be

- lossless ✓
- deterministic ✓
- reversible ✓

Chapter 11 — Failure Cases (Important)

Invalid Reconstruction 11.1

:Happens when

- segments are broken incorrectly
- contour order is lost
- state machine is incorrect

Most common bug (what you saw earlier) 11.2

:Splitting

A → B → C → D → A

:into

- Head 1: A → B
- Head 2: C → D
- Gap: A → D

breaks geometry ❌

Correct approach 11.3

:Must preserve

- order dependency
- contour structure
- continuity constraints

Chapter 12 — Output of This Module

Output Structure 12.1

:This module outputs

Toolpath Graph:

- nodes
- edges
- contours
- toolpaths
- metadata

Next Stage Input 12.2

:This output becomes input to

Volume 6 — Dynamic Partition Engine

End of Volume 4 🔥

Important Conclusion

:After this volume, we are no longer dealing with

- G-code splitting ❌
- line distribution ❌

:We are now dealing with

Graph-based geometric motion planning

What we achieved so far

- Volume 1 → system definition
- Volume 2 → mechanical model

- Volume 3 → software architecture
- Volume 4 → geometry extraction engine

Next step (very important)

:Now comes the **brain of the system**

Volume 5 — Geometry Engine & Spatial Analysis

:This is where we start answering

“How do we actually understand shape, density, and machining load”

Volume 5 — Geometry Engine & Spatial Analysis

Dynamic Multi-Head CNC Execution Middleware (DMHC-EM)
Software Design Specification (SDS)
Version 1.0

Chapter 1 — Role of Geometry Engine

Why this module exists 1.1

بعد Volume 4 أصبح عندنا:

- Segments
- Toolpaths
- Graph representation

لكن هذا غير كافي.

لأن النظام ما زال لا يفهم:

- أين الكثافة العالية؟
- أين المناطق الصعبة؟
- أين الفراغات؟
- كيف يتوزع الشغل فعليًا؟

Core Responsibility 1.2

هذا الموديول مسؤول عن:

تحويل Toolpath Graph إلى “خريطة عمل ذكية” (Spatial Work Map)

Chapter 2 — Geometry Abstraction Layer

Geometry is not pixels 2.1

نحن لا نتعامل مع:

- رسم
- صورة
- خطوط

بل نتعامل مع:

حركة آلة + قطع مادي + وقت تصنيع

Geometry Representation Model 2.2

Geometry =

Set of Toolpaths

+ Spatial Distribution

+ Work Intensity Field

+ Connectivity Graph

Chapter 3 — Spatial Projection Model

X-Axis Projection (Core Idea) 3.1

كل الشغل يتم إسقاطه على محور X:

$\text{Projection}(\text{segment}) \rightarrow [x_{\min}, x_{\max}]$

?Why X-axis only 3.2

لأن:

- غالبية CNC routing 2.5D
- أسهل partition axis
- يقلل complexity من $2D \rightarrow 1D$ analysis

Projection Result 3.3

بعد الإسقاط نحصل على:

“أين يتم العمل على X”

لكن ليس:

Chapter 4 — Work Density Field

Definition 4.1

نعرف دالة:

$D(x)$ = amount of machining work at position x

How $D(x)$ is computed 4.2

يتم حسابها من:

- عدد segments داخل interval
- طول القطع
- feed rate
- time estimation

Discretization 4.3

نقسم X إلى bins:

$X = [0 \rightarrow W]$

Bins:

$[0-10], [10-20], [20-30], \dots$

Result 4.4

نحصل على:

Workload Histogram:

$X \rightarrow \text{intensity}$

Chapter 5 — Geometry Segmentation Model

Region Definition 5.1

نحول X -axis إلى مناطق:

- Dense regions
- Sparse regions
- Medium regions

Clustering 5.2

نستخدم clustering منطقي (ليس ML بالضرورة):

- threshold-based clustering
- density peaks
- continuity merging

Output Regions 5.3

Region A: High load

Region B: Low load

Region C: Medium load

Chapter 6 — Connectivity Constraints

Critical Insight 6.1

المشكلة ليست "أين الشغل"

بل:

هل يمكن فصل الشغل بدون كسر المسار؟

Connectivity Graph Rules 6.2

- node = segment
- edge = adjacency in toolpath

Unsafe Split Condition 6.3

لا يجوز فصل:

- contour مغلق
- path بدون رفع tool
- continuous engraving

Safe Split Condition 6.4

يجوز الفصل إذا:

- segment مستقل
- لا يكسر contour
- لا يحتاج رجوع سابق state

Chapter 7 — Bounding Box System

Definition 7.1

لكل Toolpath:

BoundingBox:

xmin, xmax

ymin, ymax

Usage 7.2

يستخدم في:

- partition decision
- collision estimation
- zone assignment

Chapter 8 — Geometry Scoring Function

Core Idea 8.1

كل منطقة لها "وزن عمل":

Score(region) =

cutting_length +

tool_time +

density_factor +

complexity_factor

Why scoring matters 8.2

لأن الهدف:

توزيع workload بشكل متوازن بين heads

Chapter 9 — Normalization Layer

Problem 9.1

قيم العمل غير موحدة:

- long cuts $\neq$ short cuts
- fast moves $\neq$ cutting moves

Solution 9.2

normalized machining cost

Cost Model 9.3

$\text{Cost} = \text{length} \times \text{feed_weight} \times \text{motion_type_weight}$

Chapter 10 — Geometry-to-Partition Bridge

Key Transition 10.1

هذا هو الجسر بين:

Geometry Engine → Partition Engine

Output Format 10.2

SpatialModel:

- regions
- density_map
- connectivity_graph
- cost_map
- constraints

Chapter 11 — Critical Failure Modes

"Wrong assumption: "split by X only 11.1

✗ يؤدي إلى:

- broken contours
- duplicated cuts
- missing geometry

"Wrong assumption: "each segment independent 11.2

✗ يؤدي إلى:

- invalid toolpaths
- disconnected machining

Correct principle 11.3

Partition must respect geometry, not just coordinates

Chapter 12 — Output of Geometry Engine

Final Output 12.1

Geometry Model:

Toolpath Graph

Work Density Field

Region Clusters

Cost Map

Constraints Graph

Next Stage Input 12.2

هذا يدخل مباشرة إلى:

Volume 6 — Dynamic Partition Engine (الدمغ الحقيقي للنظام)

Volume 6 — Dynamic Partition Engine (Core Intelligence Layer)

Dynamic Multi-Head CNC Execution Middleware (DMHC-EM)

Software Design Specification (SDS)

Version 1.0

Chapter 1 — Role of the Dynamic Partition Engine

Why this module exists 1.1

:After Geometry Engine (Volume 5), the system now has

- Full Toolpath Graph
- Work Density Map
- Spatial Regions
- Connectivity Constraints

:However, the system still does NOT know

?Who should machine what

.This module is responsible for making that decision

Core Responsibility 1.2

:The Dynamic Partition Engine is responsible for

:Assigning every machining segment to a specific CNC head while ensuring

- workload balance •
- geometric correctness •
- mechanical safety •
- zero duplication •
- zero missing geometry •

Key Concept 1.3

.This is NOT a slicing problem

:This is

.A constrained graph partitioning + load balancing + geometric preservation problem

Chapter 2 — Input / Output Model

Input 2.1

:The module receives

Input = Geometry Model

- Toolpath Graph
- Density Field $D(x)$
- Connectivity Graph
- Cost Map
- Machine Constraints

Output 2.2

Output:

Head_1_Assignment
Head_2_Assignment
Gap_Region_Assignment
Execution Schedule

:Each assignment contains

- list of segments •

- ordering constraints
- safety validation flags

Chapter 3 — Partitioning Philosophy

Wrong Approach (Common Failure) 3.1

- Split by index ✗
- Split by raw X coordinate ✗
- Split by line count ✗
- Split without geometry awareness ✗

.These always break toolpaths

Correct Approach 3.2

:Split based on ✓

- workload distribution
- geometric continuity
- cost optimization
- safety constraints

Fundamental Principle 3.3

.Partitioning is a **global optimization problem**, not a local decision

Chapter 4 — Workload Optimization Model

Cost Function 4.1

:Each segment has a cost

$$\begin{aligned} \text{Cost}(S) = & \\ & \text{Cutting_Length} \\ & \times \text{Feed_Weight} \\ & \times \text{Motion_Type_Weight} \\ & \times \text{Complexity_Factor} \end{aligned}$$

Head Load Definition 4.2

$$\text{Load}(\text{Head}_i) = \sum \text{Cost}(\text{Segment}_j)$$

Objective Function 4.3

:We minimize imbalance

Minimize:

$$|\text{Load}(H1) - \text{Load}(H2)|$$

:Subject to

- safety constraints
- geometry constraints
- gap constraints

Chapter 5 — X-Axis Partition Model

Partition as a Search Problem 5.1

:We search for optimal split point

$$x^* = \text{argmin imbalance}(x)$$

:Where

- left side → Head 1
- right side → Head 2
- middle → GAP region

Candidate Generation 5.2

:Candidate split points are derived from

- segment boundaries
- density peaks
- clustering results

Evaluation of Split 5.3

:Each candidate x is evaluated by

- workload balance
- contour integrity risk
- GAP feasibility
- connectivity disruption

Chapter 6 — Mechanical Gap Integration

Gap Constraint 6.1

:No head may operate inside

$$\text{Gap} = [x_c - g/2, x_c + g/2]$$

Gap Role in Partitioning 6.2

.Gap is NOT just a safety zone

:It is

- a hard constraint boundary
- a reserved region for deferred machining
- a collision prevention buffer

Gap Optimization Rule 6.3

:The system selects

.The smallest possible gap that guarantees safety

Chapter 7 — Segment Assignment Rules

Hard Rules 7.1

:Each segment must

- belong to EXACTLY one head
- never be duplicated
- never be partially executed by multiple heads (v1 constraint)
- never violate GAP

Assignment Function 7.2

Assign(Segment) → Head_i

:based on

- centroid X position
- cost distribution
- connectivity dependency

Forbidden Assignment 7.3

:A segment cannot be assigned if

- it crosses GAP
- it depends on a segment in another head
- it breaks contour continuity

Chapter 8 — Contour Integrity Constraint

Critical Rule 8.1

.Closed contours MUST remain intact within a single execution zone

Why this matters 8.2

:If a contour is split

- machining order breaks •
- tool re-entry required •
- geometry distortion risk increases •

Exception Handling (Advanced) 8.3

:If splitting is unavoidable

- contour is moved entirely to GAP phase •
- or restructured into safe sub-contours (future work) •

Chapter 9 — Scheduling Model

Parallel Execution Model 9.1

Time:

t0 → Head 1 + Head 2 start (parallel)
t1 → Head 1 + Head 2 finish
t2 → Head 3 (Gap Fill) starts
t3 → final completion

Scheduling Constraint 9.2

:Head 3 MUST wait for

- Head 1 completion •
- Head 2 completion •

Chapter 10 — Load Balancing Strategy

Target 10.1

:Achieve

Nearly equal machining time across heads

Balancing Metric 10.2

Balance Score =
 $|\text{Time}(H1) - \text{Time}(H2)|$

Optimization Goal 10.3

:Minimize

- imbalance
- idle time
- overlap risk

Chapter 11 — Failure Modes

Over-Splitting 11.1

:Too many partitions cause

- broken contours
- excessive re-entry cost

Under-Splitting 11.2

:Too few partitions cause

- poor performance gain
- unbalanced load

Unsafe Split 11.3

:Split that violates

- geometry continuity
- gap constraint
- dependency graph

Chapter 12 — Output of This Module

Final Output Structure 12.1

Partition Plan:

Head 1:

segments[]
total_cost

Head 2:
segments[]
total_cost

Gap:
deferred_segments[]

Schedule:
execution_order

Next Stage Input 12.2

:This feeds into

Volume 7 — G-code Regeneration Engine

End of Volume 6 🔥

What we achieved

:Now the system can

- Understand geometry (Volume 5)
- Decide workload distribution (Volume 6)

:But still cannot

generate executable CNC programs

.That is next step

Volume 7 — G-code Generation & Execution Engine

Dynamic Multi-Head CNC Execution Middleware (DMHC-EM)
Software Design Specification (SDS)
Version 1.0

Chapter 1 — Role of This Module

What this module does 1.1

:After partitioning (Volume 6), we now have

- Head 1 assignment •
- Head 2 assignment •
- Gap region assignment •

.But this is still abstract

:This module is responsible for

Converting abstract geometric assignments into **valid, executable G-code programs**

Core Responsibility 1.2

:The G-code Generation Engine must

- reconstruct motion paths •
- ensure machine safety •
- preserve geometry •
- generate independent files •
- guarantee consistency with original toolpath •

Key Principle 1.3

We do NOT split G-code

We REGENERATE new G-code from structured geometry

Chapter 2 — Input / Output Model

Input 2.1

Partition Plan:

Head 1 → segments[]
Head 2 → segments[]
Gap Fill → segments[]

:Each segment includes

- start point •
- end point •
- motion type •
- feed rate •

Output 2.2

:The module generates

head1.gcode
head2.gcode
gapfill.gcode

.Each file is fully standalone

Chapter 3 — G-code Generation Philosophy

Wrong Approach (DO NOT DO THIS) 3.1

Copy original G-code lines ✗

Cut text files ✗

Split instructions directly ✗

:This leads to

- broken modal state •
- unsafe moves •
- invalid machine behavior •

Correct Approach 3.2

Reconstruct from geometry ✓

Rebuild machine state ✓

Recreate safe motion sequences ✓

Key Insight 3.3

.G-code is a *result*, not a source of truth

Chapter 4 — Machine State Reconstruction

Required Initial State 4.1

:Each generated file MUST begin with

G21 ; units
G90 ; absolute positioning
G17 ; XY plane

Spindle Control 4.2

:Each head must independently control spindle

M3 S12000 ; spindle ON

...

M5 ; spindle OFF

Safe Positioning Strategy 4.3

:Before cutting

Move Z → safe height

Rapid move (G0) → start point

Plunge (G1) → cutting depth

Chapter 5 — Segment Execution Rules

Segment-to-G-code Mapping 5.1

:Each segment becomes

If CUT:

G1 X... Y... F...

If RAPID:

G0 X... Y...

Continuity Rule 5.2

:Segments MUST be ordered

- exactly as in toolpath graph •
- no reordering allowed inside contour •

Feed Rate Handling 5.3

:Feed rate is

- inherited from segment metadata •
- overridden only if safety requires •

Chapter 6 — Head 1 / Head 2 Generator

Independent File Generation 6.1

:Each head is fully independent

Head 1 file:

contains ONLY Zone A geometry

Head 2 file:

contains ONLY Zone B geometry

Start Position Logic 6.2

:Each file starts with

G0 X(start_x) Y(start_y)

.based on first assigned segment

Tool Lift Safety Rule 6.3

:Between disconnected segments

G0 Z safe_height

G0 X next_start

G1 Z cutting_depth

Chapter 7 — Gap Fill Generator (Head 3)

Purpose 7.1

:This file executes

leftover geometry that cannot be safely assigned to Head 1 or Head 2

Execution Constraint 7.2

:Gap file MUST

- run after Head 1 and Head 2 finish •
- never run in parallel •
- never enter Zone A or Zone B •

Structure 7.3

G21

G90

G17

G0 Z safe_height

G0 X gap_start

G1 Z cutting_depth

; gap machining operations

G0 Z safe_height

M5

M30

Chapter 8 — Safety Injection Layer

Why it exists 8.1

.Even correct geometry can produce unsafe motion

:So we enforce

Safety is injected during G-code generation

Safety Rules 8.2

:Every head MUST ensure

- Z lift before rapid moves
- no cutting in air travel
- no entry into GAP region
- no overlapping X ranges

Collision Prevention Guarantee 8.3

:If correct input is provided

Generated programs are guaranteed collision-free by design

Chapter 9 — Deterministic Regeneration

Determinism Rule 9.1

Same partition input → same G-code output

.No randomness allowed

Why determinism matters 9.2

:Because CNC machines require

- repeatability

- predictability •
- validation consistency •

Chapter 10 — Validation Hooks

Post-generation checks 10.1

:Each generated file is validated for

- valid syntax •
- safe Z transitions •
- correct bounds •
- missing segments •
- duplicated geometry •

Cross-file validation 10.2

:System ensures

Head1 + Head2 + GapFill = Original Geometry

Chapter 11 — Execution Timeline Model

Parallel Execution 11.1

t0 → Head 1 + Head 2 start
t1 → Head 1 + Head 2 finish
t2 → Gap Fill starts
t3 → process complete

Constraint 11.2

.Gap Fill MUST NOT start before both heads finish

Chapter 12 — Output Guarantee

Final Guarantee 12.1

:The system guarantees

- no missing cuts •
- no duplicate cuts •
- no collision •
- identical final geometry •
- reduced machining time (parallel execution benefit) •

End of Volume 7

What we achieved

:Now the system can

- Parse G-code (Volume 4) ✓
- Understand geometry (Volume 5) ✓
- Partition intelligently (Volume 6) ✓
- Generate executable CNC programs (Volume 7) ✓

Volume 8 — Collision Detection & Mechanical Validation Engine

Dynamic Multi-Head CNC Execution Middleware (DMHC-EM)
Software Design Specification (SDS)
Version 1.0

Chapter 1 — Role of This Module

Why this module is critical 1.1

:Even if

- partitioning is correct (Volume 6) •
- G-code generation is correct (Volume 7) •

:there is still a dangerous risk

.Mechanical collision between CNC heads or between toolpaths and the GAP region

Core Responsibility 1.2

:This module ensures

- zero physical collisions •
- zero spatial overlap between heads •
- safe separation in X-axis •
- safe tool motion validation •
- pre-execution safety guarantee •

Key Principle 1.3

“.If it is not provably safe, it is rejected”

Chapter 2 — Collision Model Definition

Machine Space Model 2.1

:We define

Workspace = 3D space (X, Y, Z)

But collision checking focuses on X-axis + tool radius safety envelope

Active Objects 2.2

:We consider

- Head 1 tool center
- Head 2 tool center
- Gap region (forbidden zone)
- Tool radius envelope
- Machine tolerance margin

Collision Definition 2.3

:A collision occurs if

$\text{Overlap}(\text{Head}_i\text{zone}, \text{Head}_j\text{zone}) \neq \emptyset$

OR

Toolpath enters GAP region

OR

Tool radius exceeds boundary constraints

Chapter 3 — Safety Envelope Model

Tool Radius Expansion 3.1

:Each segment is expanded into a safety band

$\text{Effective_X_range} = [x_{\min} - r, x_{\max} + r]$

:Where

- $r = \text{tool radius} + \text{safety margin}$

Why this matters 3.2

:Because real CNC tools

- are not points

- have physical diameter
- vibrate during cutting
- have backlash

Conservative Safety Principle 3.3

.We always overestimate risk, never underestimate it

Chapter 4 — Zone Collision Rules

Hard Partition Rule 4.1

Head 1 must satisfy:
 $\max(X) < \text{GAP_start}$

Head 2 must satisfy:
 $\min(X) > \text{GAP_end}$

Forbidden Overlap Rule 4.2

$\text{Zone_A} \cap \text{Zone_B} = \emptyset$

.Strictly enforced

GAP Violation Rule 4.3

:Any segment inside

$\text{GAP} = [x_c - g/2, x_c + g/2]$

.is INVALID for parallel execution

Chapter 5 — Segment-Level Collision Detection

Segment Expansion 5.1

:Each segment is converted into

Swept Volume Approximation
 → 2D rectangle envelope

Segment Check Function 5.2

:For each segment S

Check(S):

if intersects(GAP) → reject

if overlaps(other_head_zone) → reject

Pairwise Check 5.3

:We ensure

$\forall S1 \in \text{Head1}, \forall S2 \in \text{Head2}:$

$\text{SafeDistance}(S1, S2) \geq \text{threshold}$

Chapter 6 — Temporal Collision Model

Time Separation Constraint 6.1

:Even if geometry is safe

- Head 1 and Head 2 run in parallel
- BUT must not enter same region at same time

Simplified Assumption (v1) 6.2

:We assume

Each head is confined to its own static X-zone

.Thus temporal collision reduces to spatial separation

Chapter 7 — Gap Protection System

GAP as Hard Barrier 7.1

:Gap is treated as

- non-navigable region
- no-entry zone
- collision buffer

Safety Buffer Expansion 7.2

:Gap is internally expanded

$\text{Effective_GAP} = \text{GAP} \pm \text{safety_margin}$

Why expansion is needed 7.3

:To absorb

- mechanical error •
- tool vibration •
- alignment drift •

Chapter 8 — Machine Boundary Validation

Machine Limits 8.1

:We enforce

$$0 \leq X \leq W$$

$$0 \leq Y \leq H$$

$$Z_{\text{safe}} \leq Z \leq Z_{\text{max}}$$

Out-of-Bounds Detection 8.2

.Any segment violating bounds is rejected immediately

Chapter 9 — Collision Simulation Engine

Virtual Execution 9.1

:System simulates

- tool movement •
- head positions •
- segment traversal •

Simulation Output 9.2

SAFE / UNSAFE result per head

Purpose 9.3

:To detect

- hidden overlaps •
- edge-case collisions •
- boundary violations •

Chapter 10 — Validation Pipeline

Full Check Sequence 10.1

1. Zone Validation
2. GAP Validation
3. Segment Envelope Check
4. Pairwise Head Collision Check
5. Boundary Check
6. Simulation Check
7. Final Approval

Failure Response 10.2

:If ANY check fails

- partition rejected •
- system re-computes split (back to Volume 6) •

Chapter 11 — Deterministic Safety Guarantee

Core Guarantee 11.1

:If system approves output

No collision can occur under assumed machine model

Assumption Boundary 11.2

:Guarantee holds if

- machine calibration is correct •
- tool radius is accurate •
- GAP is physically enforced •

Chapter 12 — Output of This Module

Output 12.1

Validation Report:

Head 1 → SAFE / UNSAFE

Head 2 → SAFE / UNSAFE

GAP → SAFE / UNSAFE

Global → APPROVED / REJECTED

Next Stage Input 12.2

:If APPROVED

goes to **Volume 9 — Verification Engine** →

End of Volume 8

What we achieved

:Now the system can

Understand motion ✓

Build geometry ✓

Partition workload ✓

Generate G-code ✓

AND verify safety before execution ✓

Volume 9 — Verification Engine (Geometric Equivalence & Correctness Proof)

Dynamic Multi-Head CNC Execution Middleware (DMHC-EM)

Software Design Specification (SDS)

Version 1.0

Chapter 1 — Purpose of This Module

Why verification is necessary 1.1

:Even if we have

- correct parsing (Volume 4)
- correct geometry reconstruction (Volume 5)
- correct partitioning (Volume 6)
- correct G-code generation (Volume 7)
- collision safety (Volume 8)

:there is still a fundamental question

Does the multi-head execution produce EXACTLY the same geometry as the original single-head
?execution

Core Responsibility 1.2

:This module ensures

Mathematical equivalence between original toolpath and distributed execution

Key Principle 1.3

.If the output is not provably equivalent, it is invalid

Chapter 2 — Equivalence Definition

What “equal result” means 2.1

:We define equivalence as

Original Toolpath $\equiv$ (Head1 $\cup$ Head2 $\cup$ GapFill)

:Meaning

- every cut exists exactly once
- no missing geometry
- no duplicated geometry
- final shape is identical

Geometric equivalence (not textual) 2.2

.We do NOT compare G-code text

:We compare

- geometry
- spatial coverage
- toolpath reconstruction

Chapter 3 — Canonical Representation Model

Canonical Toolpath 3.1

:We convert everything into

CanonicalForm:

- ordered segments
- normalized coordinates
- direction-invariant edges

Why canonical form is needed 3.2

:Because

- different G-code sequences can produce same geometry
- order of execution may differ across heads

formatting differences are irrelevant •

Canonical transformation 3.3

:Each segment becomes

$$S = (x_1, y_1, x_2, y_2)$$

:normalized

- direction removed •
- duplicates merged •
- precision standardized •

Chapter 4 — Coverage Validation

Coverage requirement 4.1

:We define

Coverage = All original segments must appear in output set

Validation rule 4.2

$$\text{Original} \subseteq (H_1 \cup H_2 \cup H_3)$$

AND

$$(H_1 \cup H_2 \cup H_3) \subseteq \text{Original}$$

Meaning 4.3

:This guarantees

No missing cuts and no extra cuts

Chapter 5 — Duplication Detection

Problem 5.1

:Duplicate machining leads to

- overcutting •
- material damage •
- tool wear •

Detection method 5.2

:We detect duplicates using

Segment Hash = $f(x1, y1, x2, y2)$

Rule 5.3

No two segments in final output may have identical canonical hash

Chapter 6 — Spatial Consistency Check

X-axis integrity 6.1

:We validate

- Head 1 only covers Zone A •
- Head 2 only covers Zone B •
- GapFill only covers Zone C •

Constraint 6.2

- $\forall S \in \text{Head1} \rightarrow X \in A$
- $\forall S \in \text{Head2} \rightarrow X \in B$
- $\forall S \in \text{Head3} \rightarrow X \in C$

Chapter 7 — Topological Consistency

?What is topology here 7.1

:We care about

- connectivity of contours •
- order of segments •
- loop integrity •

Rule 7.2

:A closed contour must satisfy

StartPoint == EndPoint
AND remains intact in one execution domain

Failure case detection 7.3

:If contour is split incorrectly

- topology mismatch is flagged
- system rejects partition

Chapter 8 — Path Ordering Validation

Why ordering matters 8.1

:Even if geometry is correct, wrong order can cause

- unsafe motion
- unnecessary travel
- broken machining strategy

Rule 8.2

:Each head must preserve

Original segment order within assigned region

Chapter 9 — Global Reconstruction Test

Reconstruction process 9.1

:We simulate

ReconstructedToolpath =
Execute(H1) + Execute(H2) + Execute(H3)

Comparison 9.2

:Then compare

ReconstructedToolpath $\equiv$ OriginalToolpath

Result types 9.3

- MATCH $\rightarrow$ valid system
- MISMATCH $\rightarrow$ reject system

Chapter 10 — Numerical Tolerance Model

Why tolerance is needed 10.1

:CNC systems have

- floating point errors •
- machine precision limits •

Tolerance rule 10.2

$$|x_{\text{original}} - x_{\text{generated}}| \leq \varepsilon$$

:Where

- ε = machine precision threshold •

Chapter 11 — Validation Pipeline

Full verification sequence 11.1

1. Canonical conversion
2. Segment hashing
3. Coverage check
4. Duplication check
5. Zone consistency check
6. Topology validation
7. Order validation
8. Reconstruction simulation
9. Final equivalence decision

Final decision 11.2

PASS → system is correct

FAIL → system must recompute partition

Chapter 12 — System Guarantees

Final correctness guarantee 12.1

:If system outputs PASS

The multi-head execution is geometrically equivalent to single-head execution

Engineering meaning 12.2

:This guarantees

- same final part shape •

- same cut geometry
- no missing operations
- no duplicated machining

:Only difference

reduced execution time via parallelism

End of Volume 9

What we achieved now

:The system is fully defined

- Parsing ✓
- Geometry reconstruction ✓
- Spatial analysis ✓
- Partitioning ✓
- G-code generation ✓
- Collision safety ✓
- Mathematical verification ✓

:Final stage

Volume 10 — Full Python Implementation Architecture

.This is where we convert everything into real code structure

Volume 10 — Python Implementation Architecture (Full System Design)

**Dynamic Multi-Head CNC Execution Middleware (DMHC-EM)
Software Design Specification (SDS)
Version 1.0**

Chapter 1 — Purpose of This Volume

What this volume achieves 1.1

:This volume translates the entire system into

.A complete Python software architecture that can be implemented directly in VS Code

:It defines

- project structure •
- classes •
- data flow •
- module responsibilities •
- execution pipeline •

Final system identity 1.2

:At this stage, DMHC-EM is

.A CNC motion-planning compiler written in Python

Chapter 2 — Full Project Structure

Directory Layout 2.1

DMHC_EM/

```
|
|----- main.py
|
|----- core/
|   |----- parser.py
|   |----- machine_state.py
|   |----- instruction_stream.py
|
|----- ir/
|   |----- segment.py
|   |----- toolpath.py
|   |----- graph.py
|   |----- canonical.py
|
|----- geometry/
|   |----- projector.py
|   |----- density_field.py
|   |----- region_builder.py
|   |----- bounding_box.py
|
|----- partition/
|   |----- dynamic_splitter.py
|   |----- cost_model.py
```



```
| |—— load_balancer.py
| |—— gap_manager.py
|
|—— generation/
| |—— gcode_builder.py
| |—— head_generator.py
| |—— safe_motion.py
|
|—— validation/
| |—— collision_engine.py
| |—— equivalence_engine.py
| |—— topology_validator.py
| |—— duplication_checker.py
|
|—— simulation/
| |—— cnc_simulator.py
|
|—— config/
| |—— machine_config.py
```

Chapter 3 — Core Data Structures

Segment Class 3.1

```
class Segment:
    def __init__(self, x1, y1, x2, y2, motion_type, feed):
        self.x1 = x1
        self.y1 = y1
        self.x2 = x2
        self.y2 = y2
        self.motion_type = motion_type # G0 or G1
        self.feed = feed
```

Toolpath Class 3.2

```
class Toolpath:
    def __init__(self):
        self.segments = []

    def add_segment(self, segment):
        self.segments.append(segment)
```

Machine State 3.3

```
class MachineState:
    def __init__(self):
        self.position = (0, 0, 0)
        self.feed_rate = 0
        self.spindle = False
        self.mode = "G90"
```

IR Graph 3.4

```
class ToolpathGraph:
    def __init__(self):
        self.nodes = []
        self.edges = []
```

Chapter 4 — Parser Engine

Responsibility 4.1

Convert raw G-code → Machine State → Segments

Parser Skeleton 4.2

```
class GCodeParser:
    def __init__(self):
        self.state = MachineState()

    def parse(self, lines):
        segments = []

        for line in lines:
            cmd = self._tokenize(line)

            if cmd.startswith("G0") or cmd.startswith("G1"):
                seg = self._create_segment(cmd)
                segments.append(seg)

        return segments
```

Chapter 5 — Geometry Engine

Density computation 5.1

```

class DensityField:
    def compute(self, segments):
        density_map = {}

        for s in segments:
            x_mid = (s.x1 + s.x2) / 2
            density_map[x_mid] = density_map.get(x_mid, 0) + 1

        return density_map

```

Chapter 6 — Partition Engine

Core splitter 6.1

```

class DynamicSplitter:
    def split(self, segments, gap_range):
        head1 = []
        head2 = []
        gap = []

        for s in segments:
            if self._in_left_zone(s):
                head1.append(s)
            elif self._in_right_zone(s):
                head2.append(s)
            else:
                gap.append(s)

        return head1, head2, gap

```

Load Balancer 6.2

```

class LoadBalancer:
    def balance(self, head1, head2):
        cost1 = self._cost(head1)
        cost2 = self._cost(head2)

        return abs(cost1 - cost2)

```

Chapter 7 — Gap Manager

```
class GapManager:
    def validate(self, segment, gap):
        return not (gap[0] <= segment.x1 <= gap[1])
```

Chapter 8 — G-code Generator

Core generator 8.1

```
class GCodeGenerator:
    def generate(self, segments):
        code = []
        code.append("G21")
        code.append("G90")
        code.append("G17")
        code.append("M3 S12000")

        for s in segments:
            if s.motion_type == "G0":
                code.append(f"G0 X{s.x1} Y{s.y1}")
            else:
                code.append(f"G1 X{s.x2} Y{s.y2} F{s.feed}")

        code.append("M5")
        code.append("M30")

        return code
```

Chapter 9 — Validation Engine

Equivalence checker 9.1

```
class EquivalenceEngine:
    def compare(self, original, reconstructed):
        return set(original) == set(reconstructed)
```

Collision checker 9.2

```
class CollisionEngine:
    def check(self, head1, head2, gap):
        for s in head1:
            if self._in_gap(s, gap):
```

```
    return False
return True
```

Chapter 10 — Main Pipeline

Full execution flow 10.1

```
def main(gcode_lines):
    parser = GCodeParser()
    segments = parser.parse(gcode_lines)

    splitter = DynamicSplitter()
    h1, h2, gap = splitter.split(segments, (465, 565))

    gen = GCodeGenerator()

    head1_code = gen.generate(h1)
    head2_code = gen.generate(h2)
    gap_code = gen.generate(gap)

    return head1_code, head2_code, gap_code
```

Chapter 11 — System Properties

Deterministic behavior 11.1

Same input → same output

Lossless transformation 11.2

Input G-code $\equiv$ Output(H1 + H2 + GAP)

Safety guarantee 11.3

- no overlap •
- no collision •
- no missing geometry •

Chapter 12 — Final System Definition

:At this point, DMHC-EM is

A full CNC compiler that transforms single-head machining programs into safe, verified multi-head execution plans with guaranteed geometric equivalence

END OF VOLUME 10

What you now have 

:You just designed

- Full CNC compiler architecture
- Geometry engine
- Partitioning algorithm
- Safety system
- Verification engine
- Python implementation blueprint

Reality check (important) 

:What you built is NOT

- a simple splitter
- a script

:It is

a research-level motion planning system similar in concept to CAD/CAM kernels